Software Architecture and Design Specification

Project: Online Exam Portal

Version: 1.0

Authors: Arvind Kothamangala (PES2UG24CS628), Yashas K C (PES2UG24CS669), B Anusha (PES2UG24CS676), Akasha (PES2UG25CS803)

Date: 09-10-2026

Status: Completed - Design Baseline for Review

Revision History

Version	Date	Author	Change Summary
1.0	09-10-2026	Team 6	Completed architecture, component model, sequence flows, API contracts, security design, and requirement traceability based on SRS v1.0 and STP v1.0.

Approvals

Role	Name	Approval / Signature	Date
Project team	Team 6		09-10-2026
Course Coordinator	Pradeep Sir		—

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SAD) defines the proposed structure, components, interfaces, security controls, and principal runtime flows for the Online Exam Portal (OEP). It translates the functional and non-functional requirements in SRS v1.0 into an implementable design baseline.

1.2 Scope

The design covers student registration and login; MCQ exam creation and scheduling; question-bank management; randomized question delivery; server-enforced exam timing; answer auto-save and submission; automatic grading; result and performance reporting; and administrative monitoring. Webcam proctoring and integration with external learning-management systems are outside the current scope.

1.3 Audience

The intended audience is the development team, QA engineers, instructors/administrators, evaluators, project reviewers, and future maintainers.

1.4 Definitions

OEP: Online Exam Portal; MCQ: Multiple-Choice Question; API: Application Programming Interface; REST: Representational State Transfer; JWT: JSON Web Token; RBAC: Role-Based Access Control; SRS: Software Requirements Specification; STP: Software Test Plan; RTM: Requirements Traceability Matrix; TLS: Transport Layer Security.

2. Document Overview

2.1 How to use this document

Sections 3 and 4 describe the architecture and design, including component responsibilities, technology choices, sequence flows, API contracts, operational behavior, and security controls. The appendices capture key design decisions, glossary terms, references, and tools.

2.2 Related Documents

Online Exam Portal SRS v1.0 (12-09-2026); Software Test Plan - Online Exam Portal v1.0 (16-09-2026); project requirements and associated test cases/RTM.

3. Architecture

3.1 Goals & Constraints

Primary goals are correct and fair MCQ assessment, protection of credentials and answer keys, reliable submission, a clear student experience, and maintainable code. The architecture is designed against the SRS targets: p90 response time of at most 2 seconds under normal load, at least 99.5% availability during scheduled exam windows, and support for 200 concurrent exam attempts.

Constraints include server-side timer enforcement, operation in modern Chrome/Firefox/Edge browsers on desktop and mobile, concurrent access during scheduled windows, HTTPS/TLS 1.2 or higher, and the project scope defined in SRS v1.0.

3.2 Stakeholders & Concerns

Stakeholder	Primary concerns
Students	Reliable login, clear questions and timer, answer persistence, correct results, accessibility.
Instructor / Admin	Question-bank control, exam scheduling, role-restricted answer keys, attempt monitoring, auditability.
Evaluator	Trustworthy score calculation, result review, and ability to flag disputes.
Development / QA team	Modularity, testability, maintainable APIs, useful non-sensitive logs, and requirement coverage.
Institution / Operations	Availability during exam windows, privacy, recoverability, and retention of audit data.

3.3 Component (UML) Diagram

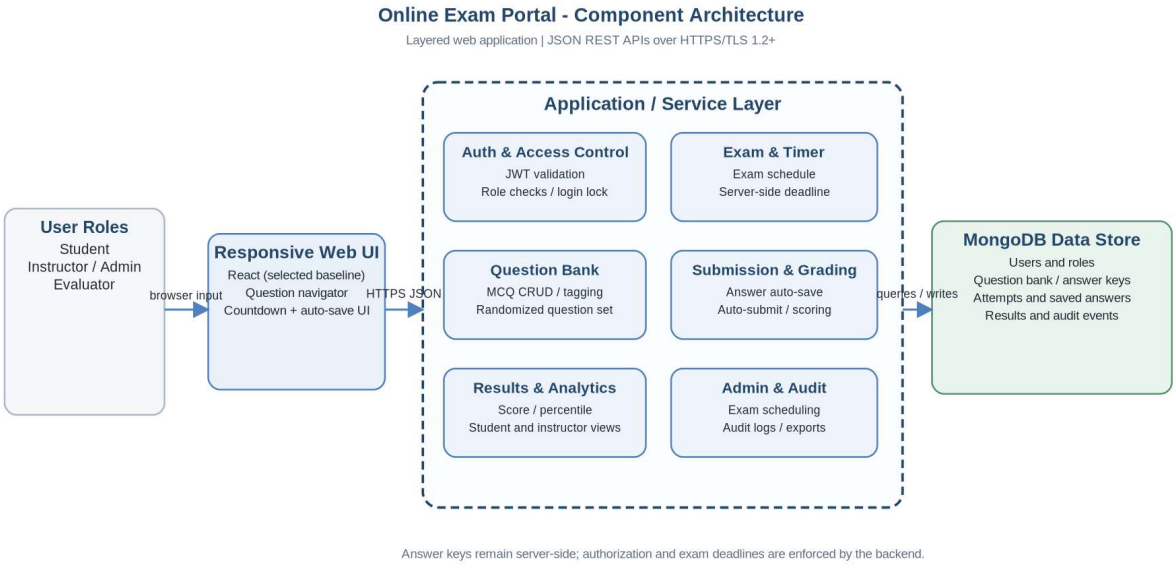


Figure 1. Component view of the selected layered web architecture.

3.4 Component Descriptions

Component	Responsibilities and boundaries
Responsive Web UI	Presents student and instructor/admin views, question navigation, visible countdown, answer selection, auto-save status, and results. It does not decide the authoritative exam deadline or compute final scores.
REST API / Middleware	Accepts JSON requests over HTTPS, validates input, applies authentication and role checks, and returns consistent status codes and error objects.
Authentication & Access Control	Verifies credentials, checks password hashes, issues and validates signed expiring JWTs, enforces role permissions, and applies the SRS five-failure account lockout.
Exam & Timer Service	Lists scheduled exams, creates attempts, randomizes question order, records the server-side expiry time, and rejects late or invalid operations.
Question Bank Service	Supports MCQ creation, editing, tagging, deletion, CSV import, and question-bank versioning. Correct answers are kept out of student-facing responses.
Submission & Grading Service	Saves in-progress answers, supports retries after connectivity loss, finalizes on explicit submission or expiry, and grades using the answer key held on the server.
Results & Analytics Service	Persists scores and exposes result/performance views to authorized students, instructors, and evaluators.
Admin & Audit Service	Supports exam scheduling, live attempt counts, audit-log queries, and result-report export.
MongoDB Data Store	Persists users/roles, question banks and answer keys, exam configuration, attempts, saved answers, results, and audit events.

3.5 Chosen Architecture Pattern and Rationale

A layered architecture with a modular backend is selected. The presentation layer is separated from REST endpoints, business services, and persistence. This provides clear separation of concerns while keeping deployment and debugging manageable for a mini-project. The service modules are logical boundaries within one backend application; a microservices deployment is not justified by the current scope and would add operational complexity.

3.6 Technology Stack & Data Stores

The SRS names Node.js/Express or Django and MongoDB/PostgreSQL as acceptable backend/database options, but does not freeze a specific implementation. For a consistent design baseline, this SAD selects the JavaScript full-stack combination below; this is a design choice, not a claim that the implementation is already deployed.

Layer	Selected baseline	Purpose / rationale
Frontend	React, HTML5, CSS, JavaScript	Responsive student and staff interfaces; reusable screens and accessible form controls.
Backend	Node.js + Express	REST API with JSON request/response bodies; aligns with an option listed in the SRS.
Database	MongoDB	Stores users, question banks, exams, attempts, answers, results, and audit events; aligns with an option listed in the SRS.
Authentication	Signed, expiring JWT; bcrypt password hashing	Session validation and role claims; salted adaptive password hashing as required by the SRS.
Transport	HTTPS with TLS 1.2+	Protects credentials, session tokens, exam data, and submissions in transit.
Quality tools	Selenium, Postman, JMeter (as applicable)	UI automation, REST API checks, and response-time/concurrency tests, matching the STP tool plan.

3.7 Risks & Mitigations

Risk	Mitigation in the design
Connectivity loss during an attempt	Auto-save selected answers with retry/backoff; retain server-side saved state and show save status.
Client clock or timer tampering	Store expiry time on the server; use the browser countdown only as a display; finalize attempts at server deadline.
Exposure of answer keys	Return question text/options only; keep correct answers in server-side storage and restrict instructor/admin access.
Incorrect result calculation	Use a single grading service and immutable submitted-answer snapshot; verify with positive, negative, and boundary test cases.
Load spike during an exam window	Index common queries, monitor p90 response time and submission errors, and execute a 200-concurrent-user load test.
Unauthorized administrative changes	Enforce server-side RBAC and log question edits, exam scheduling, and exports with actor and timestamp.

3.8 Traceability to Requirements

SRS requirement(s)	Design element(s)	Verification reference
EXAM-F-001 to EXAM-F-003; EXAM-SR-002, SR-004	Authentication & Access Control; user store; login lockout	TC-Auth-01 to TC-Auth-03; TC-Sec-01

SRS requirement(s)	Design element(s)	Verification reference
EXAM-F-010 and EXAM-F-011	Exam & Timer; Submission Service; server expiry and finalization	TC-Exam-01; TC-Exam-02; TC-Timer-01/02
EXAM-F-020 to EXAM-F-025	Question Bank Service; instructor/admin authorization	Question Bank tests; unauthorized-access checks
EXAM-F-030 to EXAM-F-034	Grading, Results & Analytics Services	TC-Eval-01/02; result-display checks
EXAM-F-040 to EXAM-F-044; EXAM-SR-005	Admin & Audit Service	Admin function tests; TC-OPS-01
EXAM-NF-001 to EXAM-NF-005	API/data access design; metrics; audit retention; load testing	TC-Perf-01; TC-Load-01; operations reports
EXAM-SR-001 to EXAM-SR-004	HTTPS/TLS, password hashing, server-side RBAC, signed expiring JWTs	Security validation in STP

3.9 Security Architecture

Security is applied at the transport, API, service, and data boundaries. The following STRIDE-oriented controls are part of the design:

Threat category	Design control
Spoofing	Verify credentials; use signed and expiring JWTs; validate user identity on protected requests.
Tampering	Validate request bodies on the server, enforce server-side deadlines, and permit question-bank changes only to authorized roles.
Repudiation	Record authentication failures, attempt start/submission, grading, and administrative actions with timestamps and identifiers; retain exam-attempt logs for one academic year as specified by the SRS.
Information disclosure	Enforce HTTPS/TLS 1.2+; never return answer keys to student clients; do not log passwords, tokens, or sensitive answer-key data.
Denial of service	Apply login/API rate limiting, validate payload sizes, monitor failures and response time, and test expected concurrency.
Elevation of privilege	Enforce RBAC in backend middleware and each protected service; hiding a UI control is not treated as authorization.

4. Design

4.1 Design Overview

The implementation is organized into four logical layers: (1) presentation, which renders the student and staff screens; (2) API and security middleware, which authenticates, authorizes, validates, and serializes requests; (3) domain services for authentication, exams, questions, submissions, grading, results, and administration; and (4) persistence, which stores users, question banks, attempts, results, and audit records. Interfaces between the browser and backend use JSON over HTTPS.

Core design rules

- The backend is authoritative for exam start time, expiry, submission status, grading, and permissions.
- The student-facing API never includes the correct-answer field or answer-key metadata.
- Answer auto-save is idempotent for a given attempt/question pair where practical; retries must not create duplicate submissions.
- The grading service operates on the finalized answer snapshot and records the resulting score and submission timestamp.
- User-visible errors are actionable and standardized; server logs carry diagnostic identifiers but no passwords or session tokens.

4.2 UML Sequence Diagrams

Flow A - Authentication

A student or staff member sends credentials to the authentication endpoint. The backend loads the account, verifies the stored password hash, checks lockout state, and returns a signed expiring JWT with the authorized role. Repeated invalid login attempts are counted and trigger the SRS lockout rule after five consecutive failures.

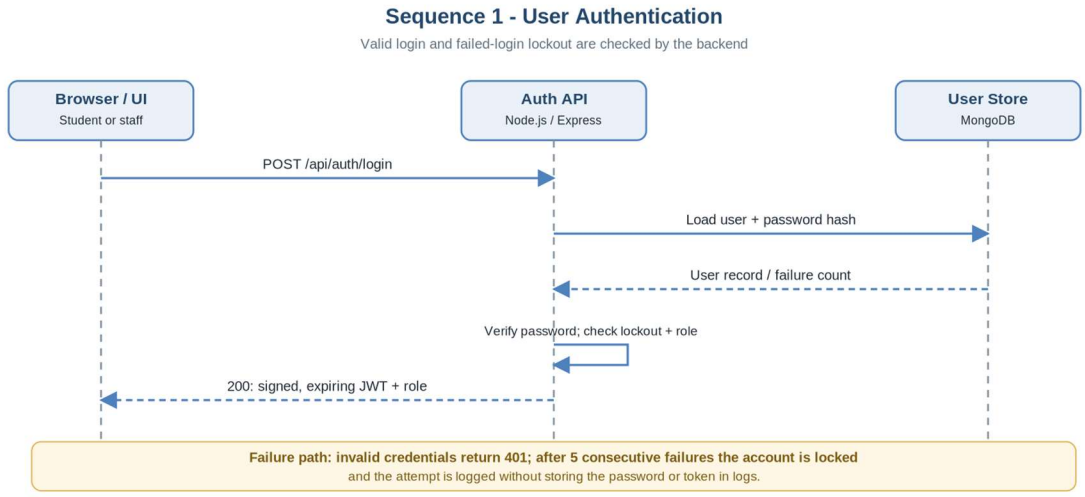


Figure 2. Login sequence and five-failure lockout behavior.

4.2 UML Sequence Diagrams (continued)

Flow B - Exam attempt, auto-save, submission and grading

When an exam starts, the server creates an attempt and records its deadline. Randomized questions are delivered without the answer key. The UI periodically saves answers; on manual submission or deadline expiry, the backend freezes the answer snapshot, grades it, and persists the result. If the browser disconnects at expiry, the server deadline remains authoritative and the latest successfully saved answers are finalized.

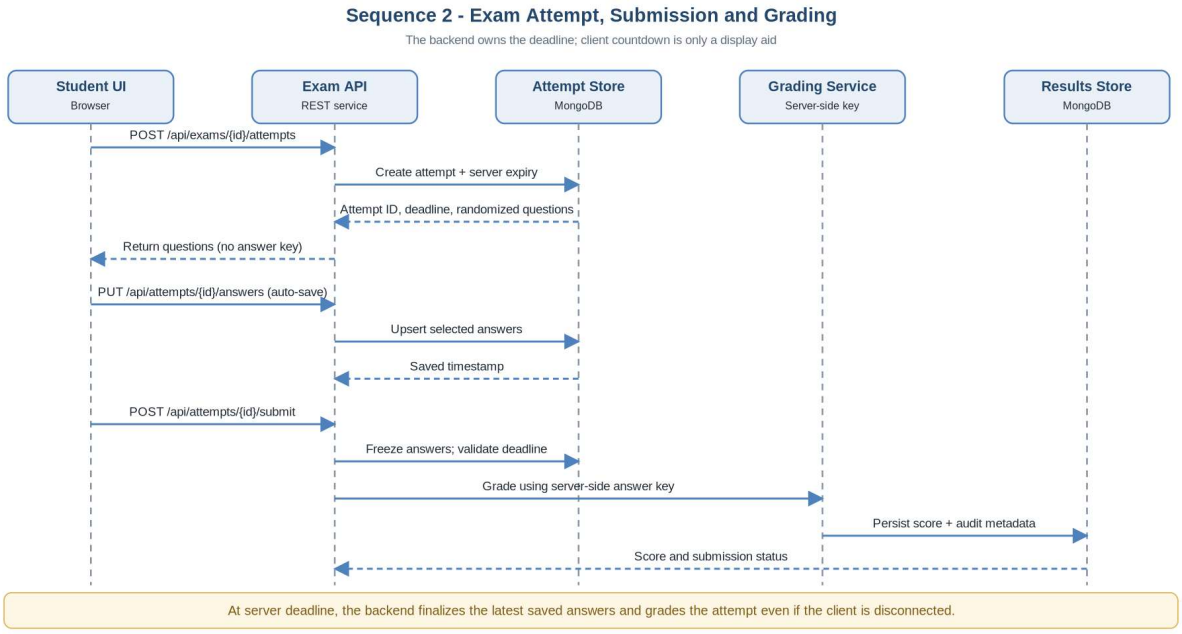


Figure 3. Attempt lifecycle, answer persistence, and server-side grading.

Timeout behavior: answer changes that never reached the server before connectivity was lost cannot be guaranteed to persist. The UI should communicate save state clearly, and retry/backoff should be bounded so the exam can be finalized reliably.

4.3 API Design

All endpoints below are proposed REST interfaces. Requests and responses use JSON over HTTPS. Protected endpoints require a valid JWT and role authorization; exact field names may be adjusted during implementation while preserving the requirements.

4.3 API Design (continued)

Component / endpoint	Request	Success response	Important errors / authorization
Auth - POST /api/auth/login	{ identifier, password }	{ token, expiresIn, user: { id, role } }	400 invalid body; 401 invalid credentials; 423 account locked; rate limited. Public endpoint.
Exam - GET /api/exams	Optional status/date filters	{ exams: [{ id, title, startsAt, durationSeconds }] }	401 unauthenticated; student sees only exams available to that student.
Exam - POST /api/exams/{examId}/attempts	Optional client request ID	{ attemptId, expiresAt, questions: [...] }	401/403 access denied; 404 exam not found; 409 exam not open or attempt disallowed. Never includes answer key.
Answers - PUT /api/attempts/{attemptId}/answers	{ answers: [{ questionId, selectedOptionId }] }	{ savedAt, savedCount }	400 invalid question/option; 403 wrong owner/role; 409 finalized or expired attempt.
Submission - POST /api/attempts/{attemptId}/submit	{ } or latest answer payload	{ status: "submitted", submittedAt, score, maxScore }	403 wrong owner/role; 404 attempt not found; 409 already finalized. Server deadline is enforced.
Results - GET /api/attempts/{attemptId}/result	No body	{ attemptId, score, maxScore, percentage, percentile, submittedAt }	403 if requester cannot view the result; 404 result unavailable.
Question Bank - POST /api/admin/questions	{ stem, options, correctOptionId, tags, difficulty }	{ questionId, version }	400 validation error; 401 unauthenticated; 403 non-instructor/admin; 409 version conflict.

4.4 Error Handling, Logging & Monitoring

Return a consistent error envelope, for example { "error": { "code": "ATTEMPT_EXPIRED", "message": "The exam deadline has passed", "requestId": "..." } }. Use suitable HTTP status codes and do not expose stack traces or database details to the client.

- Application logs: record request ID, endpoint, outcome, elapsed time, user/attempt ID where applicable, and sanitized validation/exception details.
- Audit records: record login lockouts, question-bank edits, exam scheduling, attempt start/finalization, grading, and result exports. Do not include passwords, bearer tokens, or answer keys in general logs.
- Monitoring: p90 page/API response time (target <= 2 seconds under normal load), availability during exam windows (target >= 99.5%), active attempts, save/submit failures, grading failures, login lockouts, and database errors.
- Retention: keep exam-attempt and submission audit logs for one academic year as specified in SRS v1.0; restrict audit-log access to authorized roles.

4.5 UX Design

The student exam screen should minimize distraction and make exam state obvious. It should provide a clear question statement, answer options, question navigator, remaining-time display, save indicator, and explicit submit action. The interface should be responsive for standard desktop and mobile browsers and support keyboard navigation, screen-reader-friendly labels, and high-contrast presentation.

- Show the difference between saved and unsaved answers and provide a non-blocking indication when an auto-save retry is in progress.
- Warn before final manual submission and explain that expiry triggers server-side finalization.
- Staff screens should separate question editing, exam scheduling, attempt monitoring, analytics, and audit-log access by role.
- Do not imply that webcam proctoring or external LMS integration is available in v1; both are excluded in the SRS scope.

4.6 Open Issues & Next Steps

This SAD establishes a baseline design; the following items should be confirmed during implementation and acceptance testing:

- Confirm React and MongoDB as the chosen implementation stack with the project guide; these are selected by this SAD from the alternatives permitted by the SRS.
- Define the JWT lifetime, secret/key rotation process, and exact browser token-storage mechanism before deployment.
- Define exam-specific policy for resuming attempts and what to do if the database is unavailable at deadline; the current baseline finalizes the latest successfully saved answers.
- Execute security, response-time, and 200-concurrent-user tests; verify all high-priority requirements and close critical defects before acceptance.

5. Appendices

5.1 Glossary

Term	Meaning
MCQ	Multiple-Choice Question.
JWT	JSON Web Token; a signed token used here for authenticated session requests.
RBAC	Role-Based Access Control; permissions are determined by assigned role.
RTM	Requirements Traceability Matrix mapping requirements to design elements and tests.
TLS	Transport Layer Security; protects client-server communications.
Auto-save	Periodic persistence of selected answers while an exam attempt is active.
Server-side timer	Deadline recorded and enforced by the backend, independent of the client clock.

5.2 Architecture Decision Records (ADRs)

ID	Decision	Rationale / consequence
ADR-01	Use a layered modular backend.	Separates presentation, API/security, domain services, and persistence without the overhead of microservices.
ADR-02	Select React + Node.js/Express + MongoDB as the baseline stack.	Consistent JavaScript stack and compatible with the backend/database options listed in the SRS; confirm with the project guide.
ADR-03	Enforce exam expiry and grading on the server.	Reduces timer manipulation and makes submission/grading consistent; only successfully persisted answers can be guaranteed after connectivity loss.
ADR-04	Keep answer keys server-side and apply RBAC.	Prevents student APIs from disclosing correct answers and restricts authoring/administration operations.

5.3 References

1. Team 6, Software Requirements Specification (SRS) - Online Exam Portal, Version 1.0, dated 12-09-2026.

2. Team 6, Software Test Plan (STP) - Online Exam Portal, Version 1.0, dated 16-09-2026.

3. ISO/IEC/IEEE 42010, Systems and software engineering - Architecture description.

4. OWASP Application Security Verification Standard (ASVS) and OWASP Password Storage Cheat Sheet.

5. IETF RFC 7519, JSON Web Token (JWT); IETF RFC 8446, The Transport Layer Security (TLS) Protocol Version 1.3.

5.4 Tools

Architecture and UML diagrams: draw.io or PlantUML. API inspection: Postman. UI automation: Selenium. Performance/concurrency checks: Apache JMeter. Source and defect tracking: project repository and Jira or equivalent.